Edge Computing with Kubernetes: KubeEdge in 5G, SATCOM, and Tactical Networks

Emil P. Andersen*, Ola Vanni Flaata‡ and Frank T. Johnsen*
*Norwegian Defence Research Establishment (FFI), NORWAY
‡Norwegian University of Science and Technology (NTNU), NORWAY

***Abstract*—In this paper, we discuss edge computing, arguing that Kubernetes may be a viable path to leverage modern software architecture and development methodologies as part of tactical edge computing. This research is applicable to civil-military collaboration scenarios, like disaster relief operations, as well as military defense applications. Building on previous NATO research, we specifically target one of the potentially most suitable implementations of Kubernetes to date for our experiments: KubeEdge. The paper explains our motivation, the test setup, and findings from deploying and using KubeEdge in emulated 5G, SATCOM and tactical networks.**

I. Introduction

Edge computing is a hot topic both in the defence and civilian sectors these days, and may be defined as *a distributed computing topology where information processing is located close to the edge, where things and people produce or consume that information* [1]. In other words, edge computing is about bringing the capabilities of cloud computing closer to where the data originates and where the users need the information products produced from said data. Because data does not have to travel back to a center to be processed, it enables faster analysis and more timely and relevant insights [2]. The drawback of edge computing, is that security in edge devices does not necessarily meet the same rigorous demands that many data centers have to deal with in terms of authentication, and general capabilities, which makes it a more vulnerable target than large cloud provider's data center [3]. Exploring the security perspective of edge computing, it is highlighted that *we might have to give up some of the performance increase that can come from being able to scale resources through collaboration, to protect confidentiality of processed data* [4]. Further, four properties of edge computing were identified [4]:

1) Autonomy – functionality can be maintained at the edge even without remote services.
2) Flexibility – Edge nodes can take on different functionalities to adapt to operational needs.
3) Robustness – Edge nodes can reorganize to provide network connectivity also under adverse conditions.
4) Performance – Performance can be scaled by leveraging available edge nodes.

To achieve these properties in an edge computing platform, it can be argued that these properties must be maintained as part of the Information Communications Technology (ICT) systems. While security is an important topic to address for any edge computing system, particularly related to military and disaster relief operations, we have not explored it in this study.

With the advent of modern software architectures like Microservices, it is possible to design systems that exhibit these properties [5]. Microservices coupled with modern software development methodology, DevSecOps [6], is the foundation of so-called *cloud native* [7] software development, where each service is individually developed, packaged and delivered as a software container. Cloud native means that the software is especially developed for cloud deployment, and can leverage all the elastic computing opportunities provided through the cloud. This is desirable also in edge computing, which would ensure that the same development approaches may be taken across both cloud and edge. However, with demands for flexibility, along with robustness and performance, comes complexity. To handle this complexity, a core component in edge computing is *service orchestration*. Enter Kubernetes; the container-centric management software, which has become the de facto standard to deploy and operate containerized applications [8].

In this paper, we continue the investigation started in [9] on Kubernetes and its applicability for edge computing. The results will be applicable both to military operations, as well as civil-military cooperation in operations like humanitarian assistance and disaster relief. Edge computing can provide a reliable platform for such dynamic environments. The remainder of the paper is organized as follows: Section II discusses related

work. Section III explains our motivation, use case and experiment design. Experiments and results are presented in Section IV. Section V concludes the paper.

II. Related work

Kubernetes was originally created by Google, to manage cloud computing (that is, massive data centers) computing resources [10]. Eventually, research has shown that Kubernetes also has a place outside data centers, and may be employed for edge computing. For example, in the civilian sector, Kubernetes has been explored for, e.g., Internet of Things (IoT): [11] presents a framework that uses Kubernetes and edge computing to optimize IoT service orchestration. Kubernetes manages containerized services, while edge computing handles local data processing, reducing latency and bandwidth usage. The framework improves scalability and resource optimization in IoT systems. In addition, [12] proposes a platform using Docker for containerizing machine learning (ML) apps and Kubernetes for orchestration at the edge. This architecture ensures low-latency, scalable processing of ML workloads, reducing reliance on cloud systems.

Smart cities is another area where Kubernetes may be employed, in which [13] reviews Kubernetes-based orchestration models for managing cloud and edge resources in smart cities. It discusses challenges and opportunities for improved service delivery, scalability, and low-latency processing, highlighting Kubernetes' role in distributed applications.

[14] explores Kubernetes' potential in managing edge computing environments, addressing challenges like latency, resource constraints, and intermittent connectivity. It concludes that Kubernetes, particularly lightweight distributions like K3s, can effectively orchestrate cloud-edge environments, improving scalability and performance in real-time and IoT scenarios.

KubeEdge may be used to extend the cloud to the edge, as explained in [15]. Here, the authors discuss KubeEdge's extension of Kubernetes to edge devices, and address challenges such as limited resources and intermittent connectivity by supporting offline operations, seamless integration between the cloud and edge, and real-time data processing. KubeEdge improves performance, scalability, and security in edge environments for IoT applications.

Within NATO, the research task group IST-193 titled "Edge computing at the tactical edge" has emphasis on Kubernetes and federating clusters in a multi-national context. [16] evaluates Kubernetes distributions in tactical networks with limited resources. In this study, K3s and KubeEdge performed better than K8s, with KubeEdge excelling due to local processing, making it ideal for disconnected or high-latency networks. The NATO group has also explored a use case of supporting Command and Control (C2) applications using Kubernetes [17]. Here, Kubernetes is employed in a federated cloud for C2 in tactical operations. While Kubernetes offers scalability and automation, its complexity and resource overhead pose challenges in resource-constrained environments.

These works show that while Kubernetes (and especially KubeEdge) may be well-suited to edge computing, the added constraints in employing Kubernetes in tactical networks poses several challenges. Adaptive computing has been suggested as an approach to further increase efficiency: [18] evaluates the trade-offs between performance, cost, and benefits in highly dynamic environments. Adaptive computing improves efficiency by adjusting resources in real-time, but also introduces higher operational costs and complexity, requiring careful optimization for deployment in military or disaster response scenarios. Further, [19] explains how adaptive communication is an enabler for adaptive computing.

Inspired by these works, we advance the state of the art in our study in this paper, by attempting to employ KubeEdge to roll out, update, and manage services in an ICT platform, for operational use in disaster relief and crisis operations.

III. Motivation, use case and experiment design

Kubernetes and KubeEdge serve different purposes in managing services, particularly when it comes to edge computing. Kubernetes is a container orchestration solution designed to automate and manage the deployment, scaling, and operation of containers that make up applications in clusters of nodes (see Figure 1). In a traditional Kubernetes setup, you typically have a control plane and a set of worker nodes – all such nodes may be virtual or physical servers. The main drawback of Kubernetes for edge computing is that it is originally a cloud technology, intended for massive data halls with reliable broadband connections. With mobile nodes, connectivity can be intermittent. Since Kubernetes focuses on server-based deployments, it is less ideal for managing edge devices with limited resources, battery life, or unreliable connectivity. Functionality like service discovery and networking may not work as efficiently

in highly dynamic environments, as earlier research summarized in the related work section has shown.

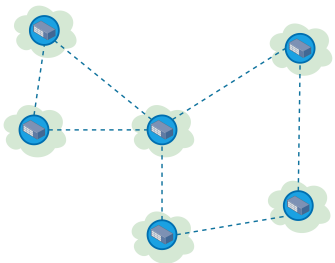

Fig. 1. Kubernetes involving mobile nodes, equally participating in a cluster.

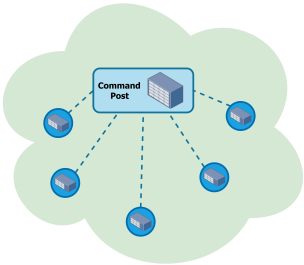

Fig. 2. KubeEdge approach, the command post contains the cloud controller, other nodes are treated as edge nodes.

KubeEdge is an extension of Kubernetes designed specifically for edge computing. It offers orchestration capabilities to edge nodes/devices through an extended Kubernetes API. The main contribution is that orchestration capabilities can be realized at edge nodes without requiring them to be continuously connected to a central cloud or data center. KubeEdge gathers control in a control plane and allows edge nodes to operate independently while still being managed from this central plane. KubeEdge is ideal for deploying applications and services to edge devices, such as IoT devices, sensors, or other resource-constrained devices, where local processing and low latency are crucial properties. This has led us to investigate KubeEdge, because this division of work in its architecture coincides with deployment where a cloudlet/private cloud would be deployed in the command post (see Figure 2). This command post may run stock Kubernetes and constitute a control plane for KubeEdge. The remaining nodes, i.e., mobile nodes parttaking in the operations, in this approach would be part of the edge devices that are controlled from the control plane. From a theoretical perspective, this is a good match to the KubeEdge architecture as well, because KubeEdge is specifically designed to manage edge devices, allowing for local processing, intermittent connectivity, and efficient rollout of services as part of edge computing. This means that KubeEdge is likely the more appropriate choice than stock Kubernetes in our platform, with edge nodes requiring local processing and independent operation.

IV. Experiment Execution and Results

A. Experiments

To model edge networks with varying constraints, several network models have been designed to replicate real-world military and NATO networks. These models aim to reflect the diverse communication needs in military operations, including integration with emerging technologies like 5G and SATCOM, applicable in civil-military collaborative operations.

For experiments conducted in the lab, we focused on three different networks that we deem relevant to the kind of deployment we discuss in this paper:

- Low-band 5G, which we will call “LAN” for simplicity in our figures. This model simulates low-band 5G networks, an area of increasing interest for military powers integrating 5G into their defense communications [20]. It offers a high data rate of up to 100 Mbps, low latency at 20 ms, and negligible packet loss (we set at 0% in the emulations).
- Tactical broadband radio, which we will refer to as “Tactical Broadband” in our figures. This technology is designed for harsh outdoor environments such as the battlefield, and the radios provide up to 2 Mbps throughput with 100 ms latency and 1% packet loss (for a typical, somewhat conservative setup – different manufacturers specifications vary – we use the values for emulation previously identified in [21]). They are optimized for secure and reliable communication, capable of covering distances up to several km in line-of-sight conditions.
- Satellite communication, commonly abbreviated “SATCOM”, is a model based on NATO’s use of SHF (Super High Frequency) and UHF (Ultra High Frequency) for satellite communications [22]. It is particularly focused on UHF, which is used for tactical communications in military operations. The data rate is relatively low (for SATCOM) at 250 kbps, with a high latency of 550 ms but usually little to no packet loss (we set at 0% in the emulations).

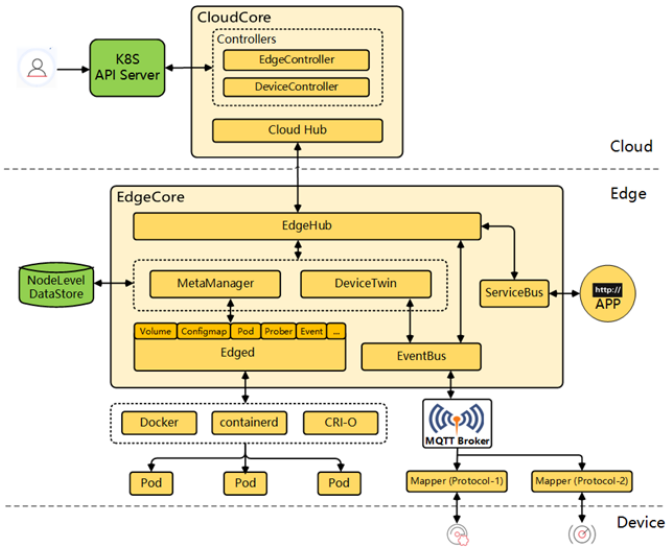

Fig. 3. KubeEdge architecture (from [23])

.

B. Testbed

KubeEdge has a two-tiered architecture, consisting of two components: CloudCore and EdgeCore, a design shown in Figure 3.

CloudCore serves as the central component of the architecture, running in the cloud or a centralized data center. It functions as the primary control plane, managing the orchestration of workloads, application updates, and overall system configuration. CloudCore integrates seamlessly with Kubernetes, leveraging its existing capabilities for container orchestration, and enables the centralized management of edge devices, ensuring consistency and scalability across distributed environments.

EdgeCore, on the other hand, operates at the edge of the network, deployed on edge devices or gateways. It acts as the local control plane, managing the local execution of workloads, device communication, and data processing. EdgeCore is responsible for ensuring that edge nodes can function independently even when disconnected from the central cloud, offering critical support for scenarios requiring low-latency processing or offline functionality. It synchronizes with CloudCore to maintain application state, enabling the seamless transition of workloads between the cloud and edge layers.

This two-tiered architecture enhances the overall efficiency and resilience of the system, as it optimizes resource usage, minimizes latency, and reduces the dependency on constant cloud connectivity, which is particularly beneficial for remote or bandwidth-constrained environments. Note that we explore bandwidth and latency as the limiting factors in this paper, CPU/memory resource constraints on edge nodes are not explored. However, a very low memory footprint is claimed, with only 70MB of memory needed to run the edge components [24].

We set up KubeEdge, using Microk8s [25] as the CloudCore. Microk8s has some overhead, we found that in its high-availability (HA) cluster mode (providing a fail-safe cluster), it exhibited a burst of 50 Mbit/s every 5 minutes to synchronize and maintain the cluster. This means, that the CloudCore, when realized with HA Microk8s, cannot and should not be attempted to run across a network with low throughput. Therefore, we foresee the use illustrated in Figure 2, where the CloudCore would reside inside the command post and being interconnected using high-throughput connections (e.g., optical fibre, mmwave, etc). Hence, the command post serves as the "cloud" side, and the tactical networks comprise the connectivity between the cloud and edge sides. MicroK8s runs in as little as 540MB of memory, but to accommodate workloads, a system with at least 20G of disk space and 4G of memory is recommended [26].

Setting up the edge side, we combined EdgeMesh with KubeEdge [27]. EdgeMesh is a container network plugin for edge nodes, enabling seamless connectivity between containers across distributed environments. Although EdgeMesh introduces some overhead, we argue that it provides critical features for mission scenarios, such as relay functionality, which allows nodes to route traffic through relay nodes when direct communication isn't possible. This enables continuous connectivity even in intermittent network conditions. Additionally, EdgeMesh enables communication between disconnected edge nodes, as long as they can detect each other, supporting communication in environments with limited or unreliable connectivity. Note that EdgeMesh relies on standard Kubernetes APIs, which means it may be deployed with any Kubernetes version, not only KubeEdge [28].

The testbed consisted of four physical nodes, of which one was used as the command post/CloudCore. The remaining three were set up to provide the edge nodes. Coupled with virtualization, we got up to 12 edge nodes total across the three physical nodes. Servers were numbered and denoted Distributed Information NOdes (DINO). The setup with only physical nodes is shown in Figure 4, and the expanded hybrid physical-virtual setup is shown in Figure 5. In addition to switching between different network models, and switching between the 4

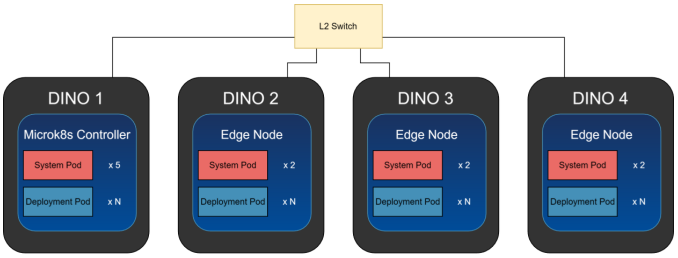

Fig. 4. Testbed diagram - showing the four physical DINO machines

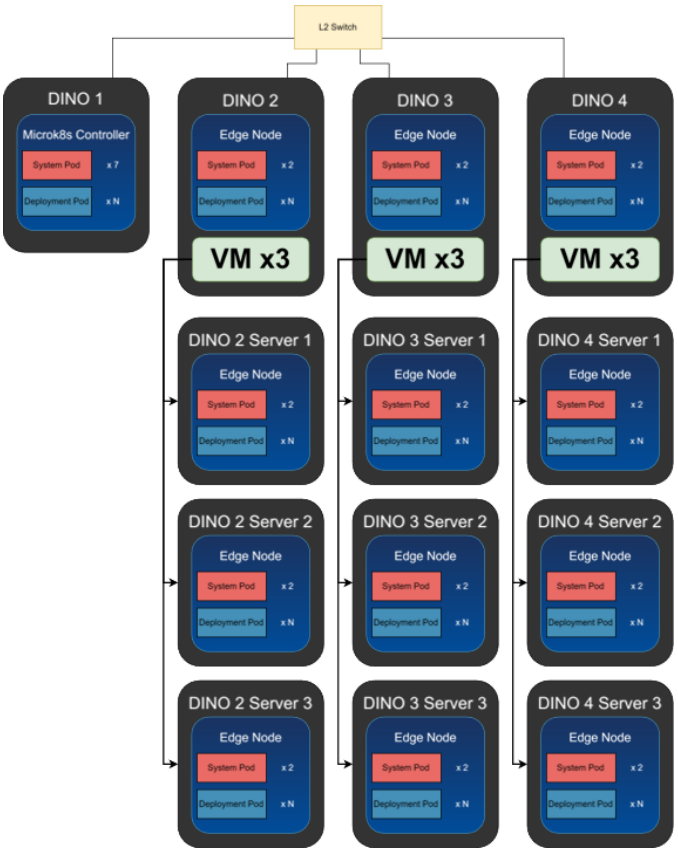

Fig. 5. Testbed diagram - hybrid setup, expanding to 12 edge nodes

nodes (1 CloudCore, 3 Edge nodes) and 13 nodes (1 CloudCore, 12 Edge nodes), the Pods (containers) to orchestrate were switched between 0/40/80 pods.

C. Results

Testing was conducted with limited throughput (see *Limitations* below), further we also tested where connection between CloudCore and Edge nodes may be lost, to mimic behavior of networks with mobile nodes (see *Disconnections* below). Finally, we investigated scaling deployments, along with rolling out updates (see *Software operations* below).

Bandwidth (outgoing) / TCP errors	0 pods (11 total)	40 pods (51 total)	80 pods (91 total)
LAN	67 kbit (42) / 0%	108 kbit (77) / 0%	151 kbit (111) / 0%
Tactical Broadband	68 kbit (43) / 2.14%	107 kbit (76) / 1.89%	150 kbit (111) / 1.68%
SATCOM	66 kbit (41) / 0.07%	105 kbit (74) / 0.20%	145 kbit (107) / 0.20%

Fig. 6. Test with four physical nodes

Bandwidth (outgoing) / TCP errors	0 pods (31 total)	40 pods (71 total)	80 pods (111 total)
LAN	419 kbit (264) / 0.08%	472 kbit (308) / 0.07%	524 kbit (351) / 0.06%
Tactical Broadband	428 kbit (268) / 3.04%	476 kbit (309) / 3.16%	528 kbit (351) / 3.25%
SATCOM	387 kbit (238) / 0.35%	385 kbit (249) / 0.26%	373 kbit* (249) / 0.33%

Fig. 7. Test with hybrid setup, note that * indicates one or more nodes becoming unavailable.

a) Limitations: The results of the testing, conducted with multiple parameters, are presented here. In the top-left corner of Figure 6, the average bandwidth measured at the controller is displayed, which includes both inbound and outbound traffic. In the bottom-right corner, the percentage of TCP errors relative to the total number of TCP packets is shown. Since the majority of traffic to Kubernetes is transmitted over TCP, this metric serves as a valuable indicator of bandwidth loss due to packet loss and other network-related effects.

Several points in the table are noteworthy. First, the number of pods is shown, which essentially represents the number of containers running within the cluster. During the execution of the test script, additional pods were created as part of the testing process. However, it is important to note that there are typically other pods already running in the cluster, which are essential for its operation or are deployed as part of Kubernetes by default. Some of these pods are also scaled in relation to the number of nodes.

The second point of interest is the bandwidth values,

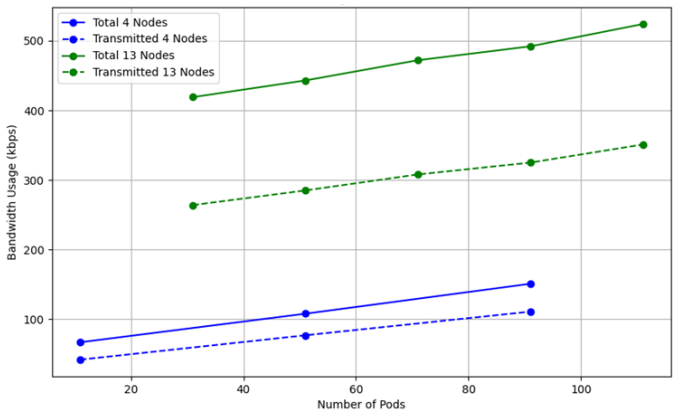

Fig. 8. Bandwidth usage vs number of pods

with the bandwidth in parentheses. The upper value indicates the total traffic captured by the controller, while the lower value represents the traffic sent by the controller.

In the following test with 13 nodes, illustrated in Figure 7, it is evident from the SATCOM data that the link is fully saturated in the middle column. The cluster operated successfully during this test, but in the test in the column to the right, nodes began to drop out of the cluster due to insufficient bandwidth to maintain connectivity with all nodes. During this test, approximately 2 to 4 nodes became unavailable.

Figure 8 illustrates how the number of nodes and pods in the cluster affects bandwidth use. The relationship appears to be relatively linear, which is consistent with expectations. The number of nodes in the cluster also significantly influences bandwidth usage. However, due to the limited number of experiments with varying node counts, it is not possible to definitively conclude whether the relationship is strictly linear. It is hypothesized that it is not perfectly linear, as this would result in zero bandwidth consumption with only two nodes. Nevertheless, if the relationship were linear, the bandwidth consumption per node would be approximately 15-25 kbps. Further, the presence of multiple points along the green line is due to tests conducted to obtain a data set that aligns with the x-axis of the last two blue points.

b) Disconnections: The first disconnection experiment involved disconnecting the controller node to observe the impact on the remaining nodes and the applications running on them. In this case, 80 copies of a deployment were running when the cable was unplugged. Figure 9 illustrates the behavior on the edge side when the controller node is disconnected. Note that the Y-axis is scaled by a factor of 10. During the first minute,

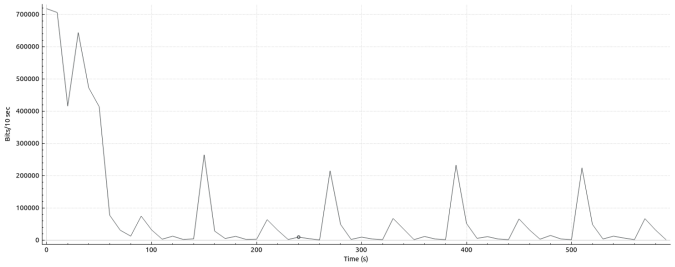

Fig. 9. Disconnecting the controller node (i.e., CloudCore)

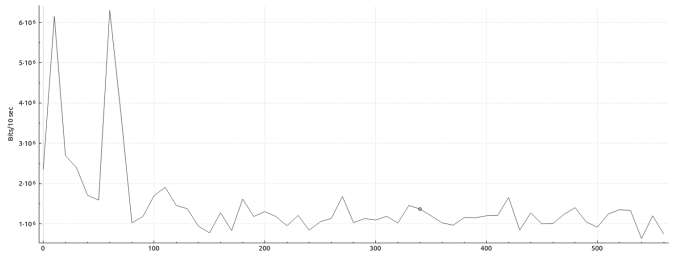

Fig. 10. Reconnecting the controller node (i.e., CloudCore)

the controller was still connected, resulting in significant traffic. However, after one minute, the controller was unplugged, and the bandwidth consumption rapidly decreased. The small peaks observed intermittently are likely due to EdgeMesh, which maintains the connection between the edge nodes. Following the disconnection of the controller, the bandwidth consumption remained very low, averaging around 3.5 kbps. After the rest of the cluster was disconnected, the controller started 60 new pods on itself, recognizing that it was short 60 pods.

Next, Figure 10 shows the behavior from the controller side when it is reconnected. While the exact nature of the traffic is unclear due to encryption, it is hypothesized that the first peak represents the rejoining of the nodes to the cluster, while the second peak corresponds to the redistribution of the pods back to the edge nodes. The final graph, Figure 11, shows the behavior on the edge node after it has been reconnected.

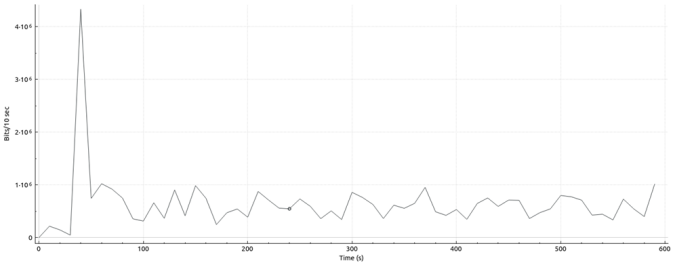

Fig. 11. Reconnected controller node (i.e., CloudCore)

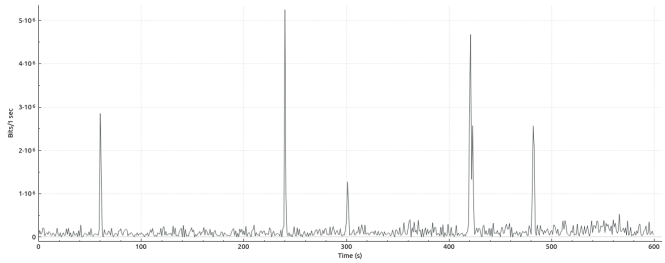

Fig. 12. Scaling in 5G

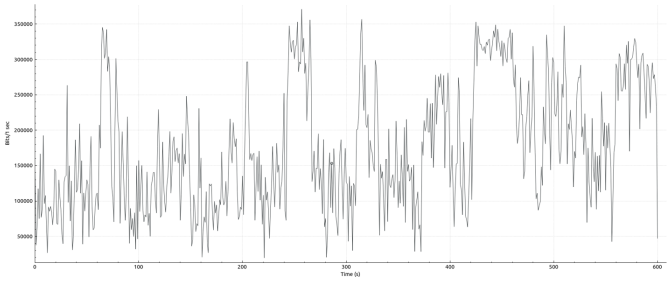

Fig. 13. Scaling in SATCOM

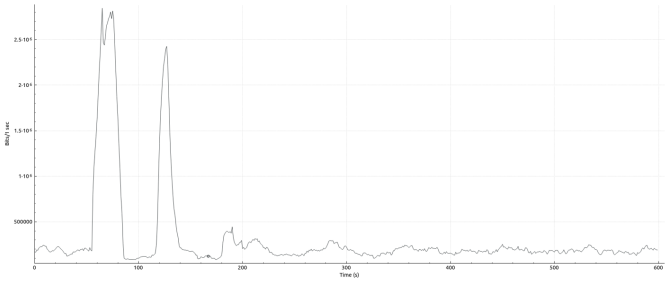

Fig. 14. Rollout in 5G

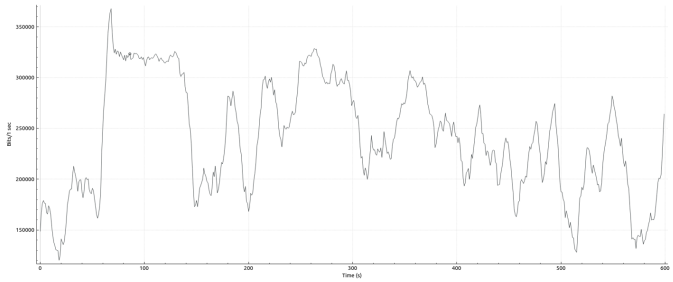

Fig. 15. Rollout in SATCOM

c) Software operations: The following illustrates the behavior when scaling deployments. Initially, a deployment with 12 pod replicas was created, followed by scaling up to 24 replicas after one minute, 48 replicas after four minutes, and 96 replicas after seven minutes. Figure 12 shows the behavior with a 5G (LAN) network.

The graph reveals a peak at each of these scaling points, as the controller spins up pods on the nodes. Additionally, there is a smaller peak approximately one minute after the second and third scaling points. While the traffic is encrypted, it is hypothesized that these smaller peaks correspond to the controller initiating pods in two batches due to the large number of replicas. In this test, approximately 550 kByte of data was used when scaling from 12 to 24 replicas, 780 kByte when scaling from 24 to 48 replicas, and 1540 kByte when scaling from 48 to 96 replicas. These two smaller peaks are excluded from the total data figures for the last two scaling steps. Although these measurements are rough estimates and include background traffic, it is estimated that approximately 60 kByte of data was used per pod when starting on an Edge node.

Figure 13 shows the same test but with the SATCOM network profile applied. Due to the bandwidth limit imposed by SATCOM, the peaks are less pronounced, but the graph still indicates where bandwidth consumption increases. In this test, all pods were successfully spun up, but the process took longer compared to the 5G (LAN) network.

Next, we tested bandwidth use during rollout of updates. Figure 14 shows the behavior on a LAN network. When updating pods in Kubernetes, the system automatically attempts to avoid updating all pods simultaneously, thus preventing downtime for services. Instead, the update is rolled out in a way that ensures no more than 25% of the pods are down at any given time. This test involved 80 pods, and the update was completed in 2 minutes and 14 seconds. Similar to the previous test, small peaks are observed one minute after the previous peaks, which is likely due to Kubernetes rolling out updates in batches with a one-minute interval between each batch.

The second graph, Figure 15 displays the same test with the SATCOM network profile applied. In this case, the update did not complete within the 10-minute trace file duration, but it was finished after 11 minutes and 28 seconds.

Results from the various rollouts:

- 13 nodes, LAN, 80 deployments: 2 minutes 30 seconds
- 13 nodes, LAN, 40 deployments: 10 seconds (first attempt), 1 minute 5 seconds (second attempt)
- 13 nodes, SATCOM, 40 deployments: 9 minutes 47 seconds
- 4 nodes, SATCOM, 40 deployments: 4 minutes 42 seconds (first attempt), 5 minutes 46 seconds (second attempt)

- 4 nodes, LAN, 40 deployments: 1 minute 10 seconds (first attempt), 1 minute 3 seconds (second attempt)
- 4 nodes, SATCOM, 80 deployments: 11 minutes 28 seconds
- 4 nodes, LAN, 80 deployments: 2 minutes 14 seconds

It should be noted, that in the tests, multicast was supported across all networks.

V. Conclusion

In this paper, we have investigated KubeEdge together with EdgeMesh as a solution for edge computing. Bandwidth consumption increases significantly with the number of nodes, particularly in relation to the number of pods. In tests involving 13 nodes, the entire SATCOM network became saturated. For SATCOM, it is advisable to use fewer than 13 nodes—approximately 8 nodes at most—depending on the required available bandwidth for other services. In contrast, with Tactical Broadband, the maximum bandwidth consumption was not reached, but it is estimated that between 20-30 nodes would represent the maximum if some bandwidth needs to be preserved for additional services. Finally, 5G would be preferable, and allow ample room for scaling up to even further nodes beyond this.

Note that for networks where multicast is not availiable, performance of EdgeMesh would be very poor. Therefore we foresee the following future work to also accommodate Kubernetes in networks lacking multicast support: 1) solving the problem by adapting the Kubernetes implementation, that is, finding a replacement network approach to EdgeMesh, or 2) exploring the concept of a "federation of clouds," where a group of smaller clouds communicate with each other as proposed by the NATO research task group IST-193 (see summary in Section II).

Based on the research in this study we find that the use of KubeEdge together with EdgeMesh is suitable for use in tactical environments. This means that the technology also has use in other areas, like civil-military collaboration and disaster relief, where communication infrastructure might be damaged or destroyed.

We acknowledge the limited testbed employed (4 physical nodes expanded to 12 virtual nodes), potentially restricting generalization to larger-scale deployments. For future work, we aim to test at a larger scale, further exploring the limits of the technology. We will also perform CPU and memory usage evaluations, as this paper only targeted the networking aspect.

References

[1] Gartner. Edge computing. [Online]. Available: https://www.gartner.com/en/information-technology/glossary/edge-computing

[2] K. Shaw, "What is edge networking and why it matters," Network World, May 31, 2022. https://www.networkworld.com/article/3224893/what-is-edge-computing-and-how-it-s-changing-the-network.html.

[3] P. Bhardwaj, "Advantage and Disadvantage of Edge Computing," https://www.tutorialspoint.com/advantage-and-disadvantage-of-edge-computing, accessed 2024-11-07.

[4] F. Mancini, "A security perspective on the military application of edge computing," NATO IST-208 Research symposium, Lillehammer, 15 October 2024.

[5] M. Alam, J. Rufino, J. Ferreira, S. H. Ahmed, N. Shah, and Y. Chen, "Orchestration of Microservices for IoT Using Docker and Edge Computing," in IEEE Communications Magazine, vol. 56, no. 9, pp. 118-123, Sept. 2018, doi: 10.1109/MCOM.2018.1701233.

[6] X. Zhao, T. Clear, and R. Lal, "Identifying the primary dimensions of DevSecOps: A multi-vocal literature review," in Journal of Systems and Software, Volume 214, 2024, 112063, ISSN 0164-1212, doi: 10.1016/j.jss.2024.112063.

[7] O. Oyeniran, A. Adewusi, A. Adeleke, L. Akwawa, and C. Azubuko, "Microservices architecture in cloud-native applications: Design patterns and scalability," *Computer Science & IT Research Journal*, vol. 5, pp. 2107–2124, 09 2024.

[8] Google, "What is kubernetes?" https://cloud.google.com/learn/what-is-kubernetes, accessed 2024-11-07.

[9] E. P. Andersen, A. Becker, O. V. Flaata, N. Jansen, and F. T. Johnsen, "Edge Computing with Kubernetes: KubeEdge in Tactical Networks," IST-209-RSY – the ICMCIS, held in Oeiras, Portugal, 13-14 May 2025 (poster).

[10] A. Verma, L. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proceedings of the European Conference on Computer Systems (EuroSys)*, Bordeaux, France, 2015.

[11] E. Kristiani, C.-T. Yang, Y. T. Wang, and C.-Y. Huang, "Implementation of an edge computing architecture using openstack and kubernetes," in *Information Science and Applications 2018: ICISA 2018*. Springer, 2019, pp. 675–685.

[12] Y. Huang, K. Cai, R. Zong, and Y. Mao, "Design and implementation of an edge computing platform architecture using docker and kubernetes for machine learning," in *Proceedings of the 3rd International Conference on High Performance Compilation, Computing and Communications*, 2019, pp. 29–32.

[13] S. Böhm and G. Wirtz, "Cloud-edge orchestration for smart cities: A review of kubernetes-based orchestration architectures," *EAI Endorsed Transactions on Smart Cities*, vol. 6, no. 18, pp. e2–e2, 2022.

[14] K. Manaouil and A. Lebre, "Kubernetes and the edge?" Ph.D. dissertation, Inria Rennes-Bretagne Atlantique, 2020.

[15] Y. Xiong, Y. Sun, L. Xing, and Y. Huang, "Extend cloud to edge with kubeedge," in *2018 IEEE/ACM Symposium On Edge Computing (SEC)*. IEEE, 2018, pp. 373–377.

[16] M. Fogli, T. Kudla, B. Musters, G. Pingen, C. Van den Broek, H. Bastiaansen, N. Suri, and S. Webb, "Performance evaluation of kubernetes distributions (k8s, k3s, kubeedge) in an adaptive and federated cloud infrastructure for disadvantaged tactical networks," in *2021 International Conference on Military Communication and Information Systems (ICMCIS)*. IEEE, 2021, pp. 1–7.

[17] M. Fogli, G. Pingen, T. Kudla, S. Webb, N. Suri, and H. Bastiaansen, "Towards a cots-enabled federated cloud architecture for adaptive c2 in coalition tactical operations: A performance analysis of kubernetes," in *MILCOM 2021-2021 IEEE Military Communications Conference (MILCOM)*. IEEE, 2021, pp. 225–230.

[18] A. Amato, H. Bastiaansen, W. Datema, M. Fogli, R. Fronteddu, R. Galliera, J. van der Geest, T. Kudla, P. Sanchez, N. Suri *et al.*, "A performance cost/benefit analysis of adaptive computing in the tactical edge," in *2024 International Conference on Military Communication and Information Systems (ICMCIS)*. IEEE, 2024, pp. 1–8.

[19] C. Barz, E. Cramer, R. Fronteddu, M. Hauge, K. Marcus, J. Nilsson, F. Poltronieri, M. Tortonesi, N. Suri, and M. Zaccarini, "Enabling adaptive communications at the tactical edge," in *MILCOM 2022-2022 IEEE Military Communications Conference (MILCOM)*. IEEE, 2022, pp. 1038–1044.

[20] S. Walker, D. Rice, M. Kahn, and J. Clark, "Why the World's Militaries are embracing 5G," https://spectrum.ieee.org/lockheed-martin-5g, accessed 2024-12-09.

[21] NATO IST-118 RTG, "SOA Recommendations for Disadvantaged Grids in the Tactical Domain," STO Open Access, Technical Report RDP, ISBN ISBN 978-92-837-2232-5, Published 5/6/2020.

[22] NATO, "Satellite communications," https://www.nato.int/cps/en/natohq/topics_183281.htm, accessed 2024-12-09.

[23] KubeEdge Project, "Why kubeedge?" https://kubeedge.io/docs/, accessed 2024-12-10.

[24] C. Mahler, "Kubernetes on the edge: getting started with kubeedge and kubernetes for edge computing," https://www.cncf.io/blog/2022/08/18/kubernetes-on-the-edge-getting-started-with-kubeedge-and-kubernetes-for-edge-computing/, 18th July 2022.

[25] Canonical Ltd., "The effortless Kubernetes: Zero-ops, pure-upstream, HA Kubernetes, from developer workstations to production," https://microk8s.io/, accessed 2024-12-10.

[26] ——, "MicroK8s – Get Started," https://microk8s.io/docs/getting-started, accessed 2025-04-10.

[27] KubeEdge Project, "Edge connection with edgemesh," https://kubeedge.io/docs/advanced/edgemesh/, accessed 2024-12-10.

[28] EdgeMesh, "Getting started," https://edgemesh.netlify.app/, accessed 2024-12-10.